

Rapport technique

Finance Data Lake

*Détection d'anomalies de marché
sur les valeurs du S&P 500*

Informations générales

Cours : Data Lakes & Data Integration
Étudiante : Akissi Aimée-Françoise
ANOUMOUYE
Formation : ING2 – Big Data & Machine Learning
Établissement : EFREI Paris
Année universitaire : 2025 - 2026

Sujet

Construction d'un data lake à trois zones (Raw / Staging / Curated) alimenté par deux sources hétérogènes, orchestré par Apache Airflow, enrichi par un autoencodeur de détection d'anomalies et exposé via une API Gateway FastAPI.

Dépôt : <https://github.com/Akissi14/finance-datalake>

12 juillet 2026

Table des matières

1	Contexte et objectifs	3
1.1	Chiffres clés du système livré	3
2	Architecture	3
2.1	Vue d'ensemble	3
2.2	Justification des trois zones	3
2.3	Modèle de données	5
2.3.1	Zone Staging — table quotes (MySQL)	5
2.3.2	Zone Curated — collection quotes_features (MongoDB)	5
2.3.3	Modèle conceptuel des données (MCD)	6
3	Sources de données	7
4	Pipelines de transformation	7
4.1	Raw → Staging	7
4.2	L'idempotence, condition de survie du lac	7
4.3	Staging → Curated : les indicateurs techniques	8
4.3.1	Définitions mathématiques	8
5	Le modèle de détection d'anomalies	10
5.1	Principe de l'autoencodeur	10
5.2	Le piège du choix des features	10
5.3	Séparation entraînement / inférence	11
5.4	Résultats	11
5.5	Validation — et une découverte	12
6	Orchestration Airflow	13
6.1	Deux réglages qui comptent	13
7	API Gateway	13
8	Niveau avancé : /ingest vs /ingest_fast	15
8.1	Un travail identique, une manière différente	15
8.2	Méthodologie de mesure	16
8.3	Le cadre théorique : la loi d'Amdahl	17
8.4	Mesures	18
8.5	Synthèse des optimisations	19
8.6	Analyse	19
8.7	Deux optimisations mesurées puis abandonnées	19
9	Difficultés rencontrées et décisions techniques	22
9.1	localstack:latest bascule en édition payante	22
9.2	torch traîne 3 Go de CUDA inutile	22
9.3	Le modèle pickle est illisible dans le conteneur	22
9.4	Un modèle non ensemencé n'est pas reproductible	22
9.5	Divergence de schéma entre l'API et le pipeline batch	23
9.6	Saturation mémoire sur 600 000 documents	23

10 Limites connues	24
11 Reproduire le projet	24
12 Conclusion	25

1. Contexte et objectifs

L'objectif est de construire un data lake complet, de l'ingestion brute jusqu'à l'exposition par API, en y intégrant une composante de *machine learning* transverse. Le domaine retenu est la **détection d'anomalies sur les marchés financiers** : identifier automatiquement, parmi des centaines de milliers de journées de cotation, celles dont le comportement s'écarte radicalement de la norme.

Principe directeur. Le code du dépôt constitue la seule source de vérité. Chaque choix documenté dans ce rapport correspond à ce qui est **effectivement implémenté et mesuré**, et non à une intention. Les chiffres cités sont ceux produits par les scripts, en suivant la procédure du §11.

Une réserve est faite sur l'entraînement du modèle : si l'initialisation du réseau est bien ensemencée (`random_state=42`), l'échantillonnage des données ne l'est pas — pour une raison documentée au §9.4. Les résultats du §5.4 valent donc pour l'exécution qui les a produits, et non à la virgule près pour une réexécution.

1.1. Chiffres clés du système livré

Indicateur	Valeur
Objets en zone Raw	685
Cotations en zone Staging	619 217
Documents enrichis en zone Curated	609 018
Valeurs boursières couvertes	505 (S&P 500)
Période couverte	2013 – 2026
Anomalies détectées	6 088 (1,00 %)
Gain de <code>/ingest_fast</code> sur 100 éléments	52,1 %

2. Architecture

2.1. Vue d'ensemble

2.2. Justification des trois zones

Zone	Techno	Justification du choix
Raw	MinIO	Le sujet impose un stockage S3 ou Elasticsearch. MinIO expose strictement la même API S3 que LocalStack (utilisé en TP), pour une image cinq fois plus légère. Le code <code>boto3</code> est identique : seul l'endpoint change. La zone conserve la donnée telle qu'ingérée , immuable, ce qui permet de rejouer tout le pipeline si la logique aval évolue.
Staging	MySQL	Les cotations OHLCV sont régulières et tabulaires. Un SGBD relationnel force le typage et permet une contrainte d'unicité (<code>ticker</code> , <code>date</code> , <code>source</code>) — clé de l'idempotence.
Curated	MongoDB	Le document <code>{ohlcv, features, metadata}</code> a pu accueillir un bloc <code>{anomaly}</code> sans migration de schéma . En SQL, il aurait fallu un <code>ALTER TABLE</code> ou une table jointe.

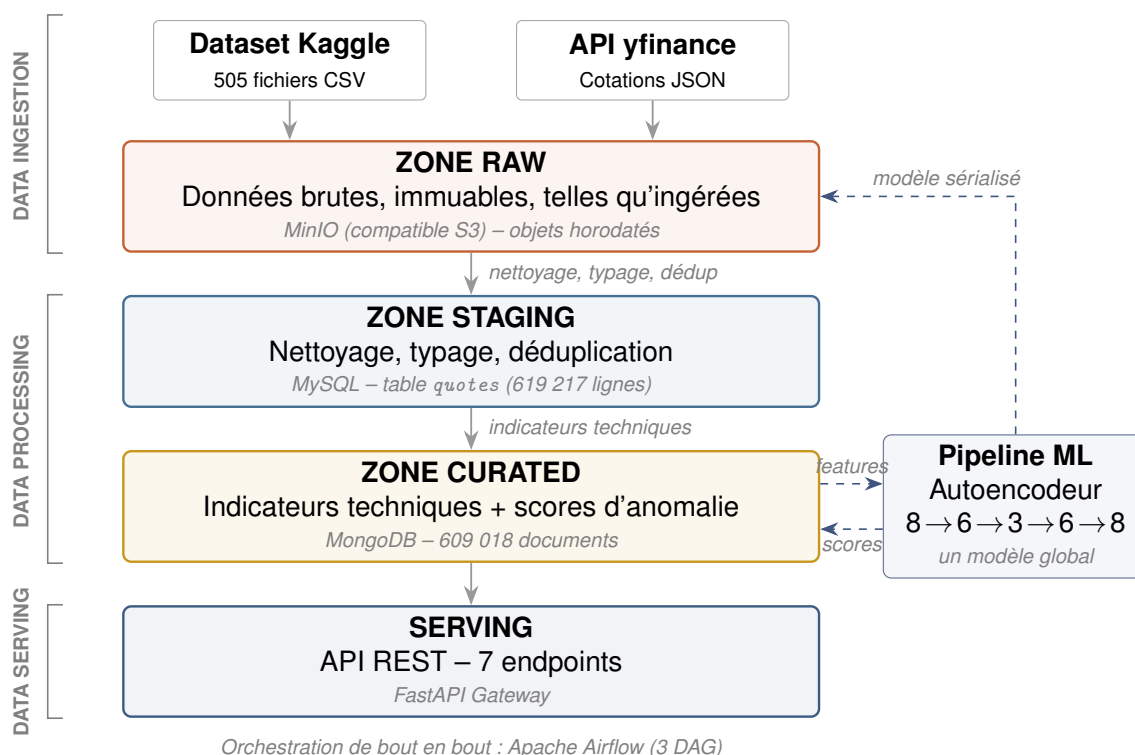


FIGURE 1 – Architecture du data lake. Les deux sources alimentent la zone Raw, qui est successivement raffinée jusqu'à la zone Curated, elle-même exposée par l'API Gateway. Le pipeline ML consomme les features de la zone Curated et y réinjecte les scores ; le modèle sérialisé est archivé dans la zone Raw.

Le modèle sérialisé (`models/autoencoder.pkl`) est stocké **dans la zone Raw**, aux côtés des données : un artefact de *machine learning* est une donnée comme une autre, et sa production doit être reproductible depuis les sources.

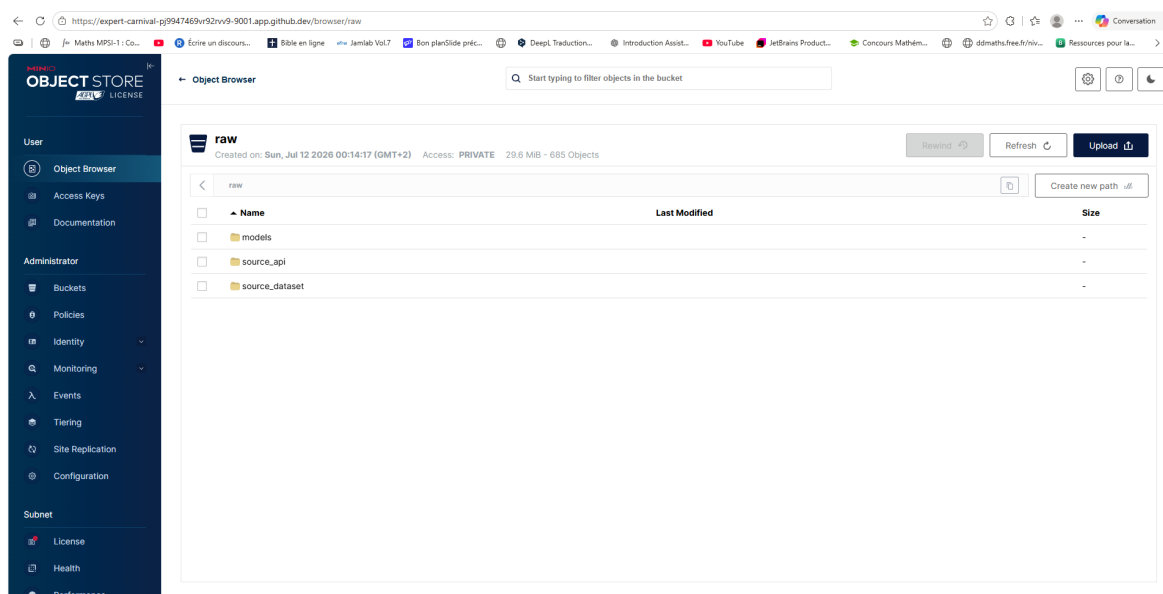


FIGURE 2 – Zone Raw : le bucket raw et ses trois préfixes — `source_dataset/` (505 CSV), `source_api/` (payloads horodatés) et `models/` (l'autoencodeur sérialisé).

2.3. Modèle de données

2.3.1. Zone Staging — table `quotes` (MySQL)

```
CREATE TABLE quotes (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  ticker      VARCHAR(10) NOT NULL, -- symbole boursier
  quote_date  DATETIME     NOT NULL, -- date de cotation
  open       DECIMAL(14,4) NOT NULL, -- cours d'ouverture
  high       DECIMAL(14,4) NOT NULL, -- plus haut du jour
  low        DECIMAL(14,4) NOT NULL, -- plus bas du jour
  close      DECIMAL(14,4) NOT NULL, -- cours de cloture
  volume     BIGINT       NOT NULL, -- volume échange
  source     VARCHAR(20) NOT NULL, -- 'dataset' ou 'api'
  loaded_at  TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,

  UNIQUE KEY uk_quote (ticker, quote_date, source),
  INDEX idx_ticker_date (ticker, quote_date)
);
```

Deux éléments méritent d'être soulignés.

La contrainte `UNIQUE (ticker, quote_date, source)` est le pivot de l'idempotence (§4.2). Le champ `source` en fait partie : une même cotation peut légitimement provenir des deux sources, et l'on souhaite conserver la traçabilité de chacune.

L'index `(ticker, quote_date)` n'est pas décoratif : c'est lui qui permet à l'API de récupérer les 20 derniers points d'une valeur sans parcourir les 619 217 lignes. Sans lui, l'endpoint `/ingest` serait inutilisable (§8.4).

2.3.2. Zone Curated — collection `quotes_features` (MongoDB)

```
{
  "ticker":    "AAPL",
  "quote_date": ISODate("2015-08-24T00:00:00Z"),
  "source":    "dataset",

  "ohlcv": {
    // la cotation d'origine
    "open": 94.87, "high": 108.80, "low": 92.00,
    "close": 103.12, "volume": 162206300
  },

  "features": {
    // ce qu'on en a dérivé
    "daily_return": -0.0212,
    "ma_5":         112.34,
    "ma_20":        119.87,
    "volatility_20": 0.0287,
    "rsi_14":       28.41,
    "volume_ratio": 3.24,
    "hl_range":     0.1629,
    "gap":          -0.0995
  },

  "anomaly": {
    // ajoute par apply_model.py, sans migration
    "score": 12.47,
```

```

    "is_anomaly": true,
    "threshold": 2.45037,
    "scored_at": "2026-07-12T13:37:47Z"
  },

  "metadata": {
    // tracabilite
    "origin": "mysql_staging",
    "computed_at": "2026-07-12T07:12:03Z"
  }
}

```

La structure imbriquée sépare explicitement **quatre natures d'information** : la cotation d'origine, ce qu'on en a dérivé, ce que le modèle en a conclu, et la traçabilité. Le bloc anomaly a été **ajouté après coup**, par un simple \$set sur des documents existants — sans migration de schéma, ni interruption de service. C'est précisément ce que le choix d'une base documentaire autorise, et ce qu'un SGBD relationnel aurait imposé de payer par un ALTER TABLE sur 609 018 lignes.

Un index unique (ticker, quote_date, source) reproduit ici la garantie d'idempotence de la zone Staging.

2.3.3. Modèle conceptuel des données (MCD)

Le modèle conceptuel décrit le domaine métier **indépendamment** de son implémentation physique. Il distingue les entités de référence (la valeur boursière, la source d'ingestion), l'entité de relevé (la cotation) et les entités dérivées (les indicateurs, le score d'anomalie). Ces dernières apparaissent en bordure pointillée : elles **enrichissent** un relevé existant et ne sont pas des données primaires.

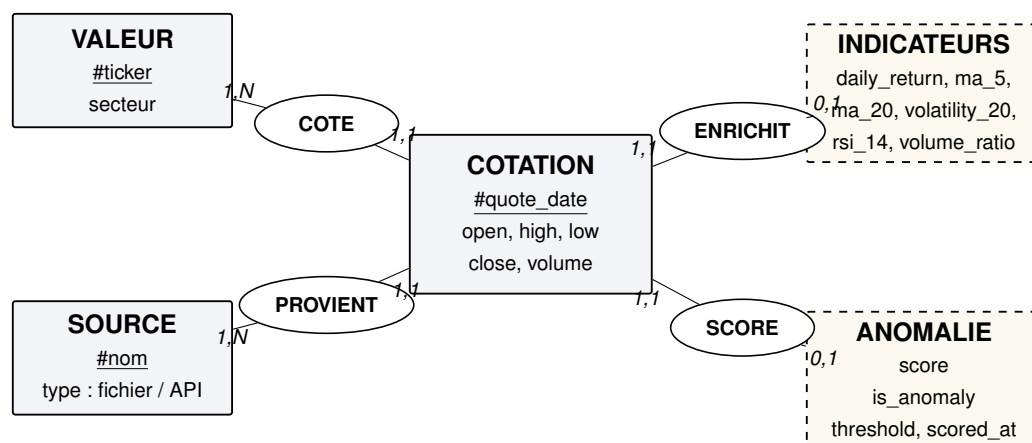


FIGURE 3 – Modèle conceptuel des données, notation Merise. Les entités en trait plein sont des données primaires ; celles en pointillé sont dérivées par les pipelines (indicateurs) ou par le modèle (score d'anomalie).

Les cardinalités traduisent des règles métier. Une **valeur** possède **plusieurs** cotations (1,N) mais une cotation appartient à **une seule** valeur (1,1). Une cotation *peut* porter des indicateurs (0,1) — les 20 premières journées de chaque valeur n'en ont pas, faute d'historique suffisant (§4.3). Elle *peut* porter un score (0,1) — seuls les documents passés par apply_model.py en possèdent un.

Le passage au modèle physique fusionne délibérément ces entités : la zone Staging les aplatit en une table unique (l'ajout d'une table valeurs pour un seul attribut serait une normalisation stérile), et la zone Curated les imbrique en un document unique (ohlcv, features,

anomaly). C'est précisément le rôle d'un modèle conceptuel : décrire le *quoi* sans préjuger du *comment*.

3. Sources de données

Le sujet exige deux sources hétérogènes.

Source	Type	Contenu	Volume
Kaggle sandp500	505 fichiers CSV	Cotations journalières du S&P 500, 2013–2018	619 040 lignes
Yahoo Finance	API REST	Cotations récentes, ré-ingérées toutes les heures	flux continu

- **Dataset Kaggle** : *S&P 500 stock data*, Cam Nugent, licence CC0 1.0 — <https://www.kaggle.com/datasets/camnugent/sandp500>
- **API** : Yahoo Finance, via la bibliothèque `yfinance` — <https://pypi.org/project/yfinance/>

Les deux sources convergent sur le **même schéma** (date, open, high, low, close, volume, ticker), ce qui permet de les faire atterrir dans une **table Staging unique**. C'est précisément le rôle de cette zone : unifier l'hétérogène.

4. Pipelines de transformation

4.1. Raw → Staging

Le script `load_to_staging.py` assume quatre responsabilités : unifier, typer, nettoyer, dé-dupliquer. Six règles de nettoyage sont appliquées, chacune comptée et journalisée — un nettoyage silencieux est un nettoyage qu'on ne peut pas auditer.

```
lignes brutes          : 619040
apres valeurs manquantes : 619029 (-11)
apres typage           : 619029 (-0)
apres prix invalides   : 619029 (-0)
apres incoherences OHLC : 619017 (-12)
apres doublons         : 619017 (-0)
```

Les **12 incohérences OHLC** correspondent à des lignes où le plus haut du jour était inférieur au plus bas — physiquement impossible. Ce sont de vrais défauts du jeu de données source, détectés par le pipeline.

4.2. L'idempotence, condition de survie du lac

Airflow relance ce script **toutes les heures**. Un pipeline qui duplique ses données à chaque exécution transforme un data lake en *data swamp* en quelques jours.

La solution repose sur une contrainte `UNIQUE (ticker, quote_date, source)` en base, combinée à `INSERT IGNORE`. Le rejeu est donc **démontrable** :


```
$ python src/load_to_staging.py --source both # 1er run
Lignes inserees ce run : 619217

$ python src/load_to_staging.py --source both # 2e run
Lignes inserees ce run : 0
```

La même logique est reproduite en zone Curated via un index unique MongoDB et `ordered=False`.

4.3. Staging → Curated : les indicateurs techniques

Feature	Ce qu'elle capture
daily_return	variation quotidienne du cours de clôture
ma_5, ma_20	moyennes mobiles court et moyen terme
volatility_20	écart-type glissant des rendements — cœur de la détection
rsi_14	momentum [4] : sur-achat (>70) / sur-vente (<30)
volume_ratio	volume du jour / sa moyenne — un pic précède souvent un krach
hl_range	amplitude intra-journalière, normalisée par le prix
gap	écart entre l'ouverture et la clôture de la veille

4.3.1. Définitions mathématiques

Soit C_t le cours de clôture au jour t , V_t le volume, H_t et L_t les extrêmes du jour, O_t l'ouverture. On note r_t le rendement quotidien.

$$r_t = \frac{C_t - C_{t-1}}{C_{t-1}} \quad \text{rendement quotidien} \quad (1)$$

$$MA_t^{(n)} = \frac{1}{n} \sum_{i=0}^{n-1} C_{t-i} \quad \text{moyenne mobile sur } n \text{ jours} \quad (2)$$

$$\sigma_t = \sqrt{\frac{1}{n-1} \sum_{i=0}^{n-1} (r_{t-i} - \bar{r})^2} \quad \text{volatilité glissante, } n = 20 \quad (3)$$

$$RSI_t = 100 - \frac{100}{1 + RS_t}, \quad RS_t = \frac{\bar{G}_t}{\bar{P}_t} \quad \text{momentum, } n = 14 \quad (4)$$

où $\bar{G}_t$ et $\bar{P}_t$ sont les moyennes des gains et des pertes sur la fenêtre. Le RSI est borné dans $[0, 100]$ par construction : au-delà de 70 la valeur est dite sur-achetée, en dessous de 30 sur-vendue.

Application numérique. Le document témoin de la zone Curated (Apple, 24 août 2015) porte $\bar{G} = 0,0084$ et $\bar{P} = 0,0212$ sur la fenêtre de 14 jours. Il vient

$$RS = \frac{0,0084}{0,0212} = 0,396, \quad RSI = 100 - \frac{100}{1,396} \approx 28,4 \quad (5)$$

soit exactement la valeur `rsi_14 = 28,41` portée par le document, et une valeur nettement **sur-vendue** — cohérent avec une journée de krach.

$$VR_t = \frac{V_t}{\frac{1}{n} \sum_{i=1}^n V_{t-i}} \quad \text{ratio de volume} \quad (6)$$

$$HL_t = \frac{H_t - L_t}{C_t} \quad \text{amplitude intra-journalière} \quad (7)$$

$$g_t = \frac{O_t - C_{t-1}}{C_{t-1}} \quad \text{gap d'ouverture} \quad (8)$$

Ordre de grandeur de la volatilité. La volatilité σ_t est *quotidienne*. Pour la comparer aux chiffres usuels du marché, on l'annualise en supposant les rendements indépendants, ce qui autorise la mise à l'échelle en racine du nombre de séances (252 par an) :

$$\sigma_{\text{ann}} = \sigma_t \sqrt{252} \quad (9)$$

La volatilité quotidienne moyenne mesurée sur la zone Curated vaut $\sigma_t = 0,0142$, soit

$$\sigma_{\text{ann}} = 0,0142 \times \sqrt{252} \approx 0,225 = 22,5 \% \quad (10)$$

un ordre de grandeur conforme à celui d'un indice actions développé. Le calcul est donc plausible, et non seulement syntaxiquement correct.

Point de conception. Les indicateurs sont calculés **par couple (ticker, source)**, jamais par ticker seul. Le dataset s'arrête en 2018 et l'API reprend en 2026 : une moyenne mobile calculée à cheval sur ces deux périodes enjamberait un trou de huit ans et produirait des valeurs dénuées de sens.

Les 20 premières lignes de chaque groupe (10 199 au total) sont écartées : elles n'ont pas l'historique nécessaire à une fenêtre de 20 jours. Choix assumé — une moyenne mobile calculée sur 3 points ne serait pas comparable à une moyenne sur 20. Le compte est cohérent : $619\,217 - 10\,199 = 609\,018$ documents en zone Curated.

5. Le modèle de détection d'anomalies

5.1. Principe de l'autoencodeur

Un autoencodeur est un réseau de neurones entraîné à **reproduire son entrée en sortie**, en la faisant transiter par une couche cachée plus étroite :

$$8 \text{ features} \rightarrow 6 \rightarrow 3 \rightarrow 6 \rightarrow 8 \text{ features}$$

Le goulot à 3 neurones le contraint à apprendre la structure du comportement **normal**. Sur une journée ordinaire, il reconstruit fidèlement : l'erreur est faible. Sur un krach flash, une bulle de volume, une volatilité aberrante, il n'a jamais rien vu de tel : l'erreur explose. **L'erreur de reconstruction est le score d'anomalie.**

Formellement, soit $\mathbf{x} \in \mathbb{R}^8$ le vecteur de features standardisé d'une journée. L'autoencodeur apprend deux fonctions :

$$\underbrace{f_\theta : \mathbb{R}^8 \rightarrow \mathbb{R}^3}_{\text{encodeur}} \quad \text{et} \quad \underbrace{g_\phi : \mathbb{R}^3 \rightarrow \mathbb{R}^8}_{\text{décodeur}} \quad (11)$$

en minimisant l'erreur de reconstruction sur l'ensemble d'entraînement :

$$(\theta^*, \phi^*) = \arg \min_{\theta, \phi} \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}_i - g_\phi(f_\theta(\mathbf{x}_i))\|_2^2 \quad (12)$$

Le **score d'anomalie** d'une observation est alors son erreur de reconstruction :

$$s(\mathbf{x}) = \frac{1}{8} \sum_{j=1}^8 (x_j - \hat{x}_j)^2, \quad \hat{\mathbf{x}} = g_{\phi^*}(f_{\theta^*}(\mathbf{x})) \quad (13)$$

et l'observation est déclarée anormale si $s(\mathbf{x}) > \tau$, où le seuil τ est le quantile d'ordre $1 - \alpha$ des erreurs d'entraînement, α étant le taux de contamination attendu :

$$\tau = Q_{1-\alpha}(\{s(\mathbf{x}_i)\}_{i=1}^N), \quad \alpha = 0,01 \quad (14)$$

L'implémentation repose sur `MLPRegressor` de `scikit-learn` [5], entraîné avec $\mathbf{X}$ en entrée **et** en cible — c'est ce qui en fait un autoencodeur.

5.2. Le piège du choix des features

Les indicateurs de la zone Curated ne sont **pas tous utilisables tels quels**. `ma_5` et `ma_20` sont des **prix absolus** : Amazon cote plusieurs milliers de dollars, Coca-Cola quelques dizaines. Un modèle global apprendrait surtout « Amazon est chère », pas « cette journée est anormale ».

Huit features **sans échelle** sont donc dérivées : rendements, ratios, indicateurs bornés (`ma_ratio` = `ma_5/ma_20`, `price_to_ma20` = `close/ma_20`, `rsi/100`...). C'est la condition pour qu'un modèle unique ait du sens sur les 505 valeurs.

Une **standardisation** précède l'entraînement : sans elle, la volatilité ($\approx 0,01$) pèserait mille fois moins que le `volume_ratio` (≈ 1) dans l'erreur quadratique, et le modèle serait aveugle au signal recherché.

5.3. Séparation entraînement / inférence

Deux scripts distincts, comme en production :

- `train_model.py` — entraîne sur un **échantillon** de 100 000 documents, calcule le seuil, sérialise vers MinIO. Tourne en @weekly.
- `apply_model.py` — recharge l'artefact et score l'**intégralité** des 609 018 documents. Tourne à chaque pipeline.

5.4. Résultats

```
Convergence en 40 epoques
Erreur de reconstruction moyenne : 0.25542
Erreur mediane                  : 0.10550
Erreur maximale                  : 68.06818
Seuil (contamination 1 %)      : 2.45037
Anomalies detectees             : 6 088 / 609 018 (1.00 %)
```

Ces chiffres sont ceux d'une **exécution donnée**. L'échantillon d'entraînement étant tiré par l'opérateur `$sample` de MongoDB, qui n'est pas ensemencable (§9.4), une réexécution produit des valeurs *voisines* mais non identiques : le seuil oscille autour de 2,45 et le nombre d'anomalies autour de 1 % des documents. Cette stabilité est cohérente avec le taux de contamination retenu, et constitue en soi une vérification — mais elle ne dispense pas de la correction identifiée au §9.4.

Lecture du seuil. Le seuil $\tau = 2,450$ se situe à

$$\frac{\tau}{\tilde{s}} = \frac{2,450}{0,1055} \approx 23 \quad (15)$$

fois l'erreur *médiane* $\tilde{s}$. Autrement dit, une journée n'est déclarée anormale que si le modèle la reconstruit **vingt-trois fois plus mal** qu'une journée ordinaire. Le pire cas observé sur l'ensemble de la zone Curated (XL, 24 août 2015, $s = 321,8$) se situe, lui, à

$$\frac{321,8}{0,1055} \approx 3\,050 \quad (16)$$

fois l'erreur médiane. Sur le seul échantillon d'entraînement, le rapport entre l'erreur maximale et l'erreur médiane atteint déjà $68,07/0,1055 \approx \mathbf{645}$: c'est la signature recherchée. La grande majorité des journées se ressemblent, une poignée est radicalement hors norme.

Sur l'évaluation. La détection étant **non supervisée**, aucune étiquette de vérité terrain n'est disponible : une matrice de confusion n'est donc pas calculable, et il serait malhonnête d'en produire une. L'évaluation repose sur trois éléments : le taux de contamination retenu (une hypothèse métier, assumée comme telle), la forme de la distribution des scores (une masse concentrée autour de la médiane, une queue lourde au-delà du seuil), et la **confrontation des anomalies remontées à des événements de marché connus** — ce que fait la section suivante.

5.5. Validation — et une découverte

Les dix journées les plus anormales de la zone Curated, telles que remontées par `apply_model.py` :

Ticker	Date	Score	Rendement	Vol/moy
XL	2015-08-24	321,8	−3,6 %	1,6
NI	2015-07-02	127,2	−62,6 %	5,0
EBAY	2015-07-20	122,5	−56,9 %	2,7
HCA	2015-08-24	97,5	−2,3 %	2,1
BAX	2015-07-01	79,9	−44,4 %	2,8
DISCA	2014-08-07	78,9	−50,6 %	1,9
DISCK	2014-08-07	77,7	−51,0 %	4,5
SYN	2013-12-09	76,1	+9,7 %	11,9
AOS	2017-07-25	70,9	−0,5 %	14,1
LNT	2016-06-30	69,0	−0,3 %	13,9

Validation du modèle

Deux valeurs, la même date : XL et HCA, le 24 août 2015. C'est le *flash crash* du 24 août 2015 — le Dow Jones a perdu 1 000 points à l'ouverture. Le modèle l'a retrouvé **seul**, sans qu'on lui ait jamais indiqué qu'une telle journée existait. XL occupe la première place du classement, avec un score trois mille fois supérieur à l'erreur médiane.

Découverte : un défaut du jeu de données source

Les rendements à −62 %, −57 %, −44 % et −51 % ne sont **pas** des anomalies de marché. Vérification faite :

- NI, 02/07/2015 : NiSource **scinde** Columbia Pipeline Group
- EBAY, 20/07/2015 : eBay **détache** PayPal
- BAX, 01/07/2015 : Baxter **scinde** Baxalta
- DISCA/DISCK, 07/08/2014 : **split** d'actions

Ce sont des **opérations sur titres** — et elles occupent **cinq des dix premières places** du classement. Le prix chute mécaniquement. **Le dataset Kaggle n'est pas ajusté des splits** — ce que sa description ne mentionne pas. Le pipeline a donc mis au jour un défaut de qualité de la source.

Correction envisagée : filtrer les variations > 30 % non confirmées par un pic de volume, ou croiser avec un référentiel d'opérations sur titres.

Enfin, les valeurs les plus souvent anormales sont CHK (Chesapeake Energy, 209 journées), AMD (179), FCX (Freeport-McMoRan, 150), WMB (Williams, 101) et MRO (Marathon Oil, 101). Quatre d'entre elles sont des valeurs de l'énergie ou des matières premières, précisément les plus exposées à **l'effondrement pétrolier de 2014–2016**. AMD relève d'une autre cause : le titre a connu sur la période une volatilité propre exceptionnelle, sans rapport avec le pétrole. Le modèle ne fait pas la différence entre ces deux régimes — il signale l'écart à la norme, pas sa cause.

6. Orchestration Airflow

Trois DAG, dont la séparation traduit **trois rythmes différents** :

DAG	Cadence	Rôle
finance_datalake_pipeline	manuel	Pipeline complet : les 2 ingestions en parallèle , puis Staging → Curated → inférence
finance_api_scheduled	@hourly	Ré-ingestion périodique de l'API — exigence explicite du sujet
finance_train_model	@weekly	Ré-entraînement de l'autoencodeur

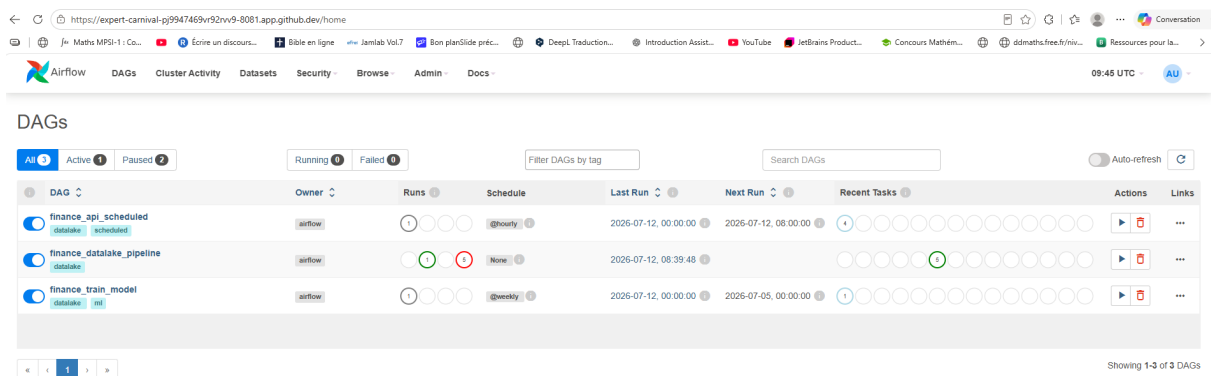


FIGURE 4 – Les trois DAG chargés par le scheduler. `finance_api_scheduled` est actif et se déclenche toutes les heures.

Pourquoi trois DAG et non un seul ? Parce que l'entraînement est coûteux et l'inférence légère. Ré-entraîner un autoencodeur toutes les heures serait aussi absurde que de ne jamais le ré-entraîner.

Airflow orchestre, il ne calcule pas. Chaque tâche se contente de lancer le script correspondant via un `BashOperator`. La logique métier reste dans `src/`, testable et rejouable **en dehors** d'Airflow. Un DAG qui contient de la logique métier est un DAG qu'on ne peut ni tester ni réutiliser.

6.1. Deux réglages qui comptent

- `catchup=False` — sans lui, Airflow rejouerait *tous* les créneaux horaires depuis le `start_date`, soit des milliers d'exécutions d'un coup.
- `-period 5d` dans le DAG horaire (contre `1mo` dans le complet) — on ne re-télécharge pas un mois d'historique toutes les heures, et la clé d'unicité écarte de toute façon ce qui est déjà connu.

7. API Gateway

FastAPI, sept endpoints, documentation interactive auto-générée sur `/docs`.

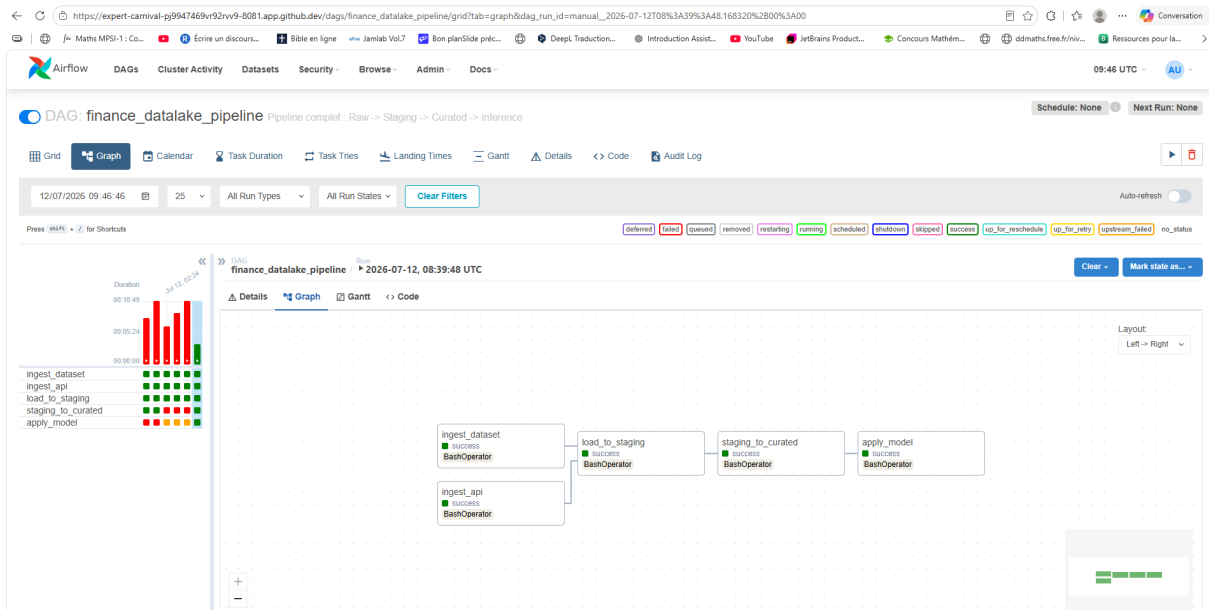


FIGURE 5 – Exécution complète du DAG `finance_datalake_pipeline`. Les deux ingestions s'exécutent **en parallèle** avant de converger sur `load_to_staging`.

Endpoint	Méthode	Rôle
/	GET	Carte des endpoints
/health	GET	État des trois zones + du modèle
/stats	GET	Volumétrie de remplissage des trois zones
/raw	GET	Inventaire des objets MinIO (filtrable par préfixe)
/staging	GET	Cotations MySQL (filtrable, paginé)
/curated	GET	Documents enrichis (filtre <code>anomalies_only</code>)
/ingest	POST	Ingestion à chaud — implémentation de référence
/ingest_fast	POST	Ingestion à chaud — implémentation optimisée

Les filtres SQL passent par des **requêtes paramétrées** (%s), jamais par concaténation : une concaténation ouvrirait la porte à l'injection SQL.

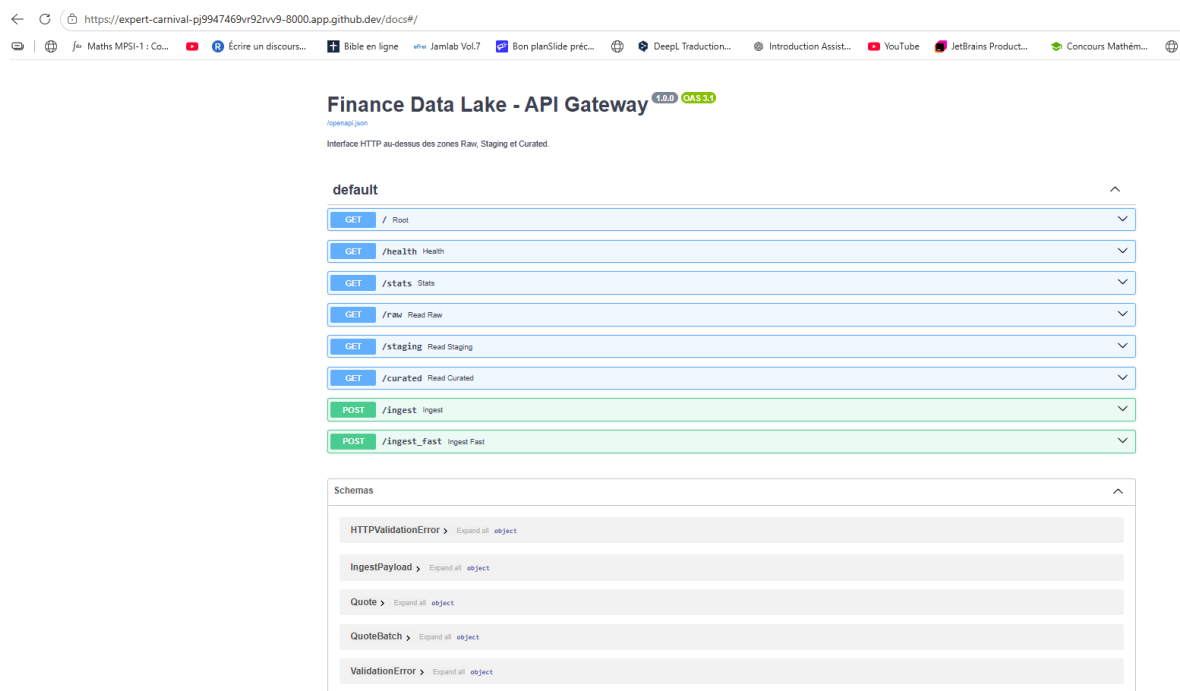


FIGURE 6 – Documentation interactive générée par FastAPI (/docs). Chaque endpoint est testable directement depuis le navigateur.

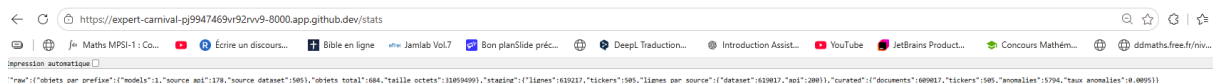


FIGURE 7 – Réponse de /stats : la volumétrie réelle des trois zones, interrogée en direct.

8. Niveau avancé : /ingest vs /ingest_fast

8.1. Un travail identique, une manière différente

Les deux endpoints effectuent **rigoureusement le même travail** : valider le lot (Pydantic), archiver le payload brut en zone Raw, récupérer l'historique récent de chaque valeur, calculer les 8 features, scorer avec l'autoencodeur, écrire en zone Curated.

	/ingest	/ingest_fast
Archivage	bloquant, avant tout calcul	concurrent (ThreadPoolExecutor)
Historique	1 requête Mongo par cotation	mêmes requêtes indexées, en parallèle
Scoring	1 predict() par ligne	1 predict() sur la matrice
Erreur de reconstruction	boucle Python interprétée	Numba (@njit(parallel=True))
Écriture	1 update_one par document	1 bulk_write

POST /ingest_fast Ingest Fast

Ingestion a chaud : IMPLEMENTATION OPTIMISEE.

Le resultat est rigoureusement identique a /ingest. Cinq optimisations :

1. ARCHIVAGE CONCURRENT. L'écriture du payload dans MinIO est un aller-retour réseau d'une quinzaine de millisecondes, totalement independant du calcul qui suit. On la lance dans un thread et on ne l'attend qu'a la toute fin : les deux travaux se recouvrent au lieu de s'additionner. Decisif sur les petits lots, ou les couts fixes dominent.
2. HISTORIQUES EN PARALLELE. Memes requetes indexees que la baseline, mais emises concurremment (voir _fetch_histories, qui documente aussi une optimisation tentee puis abandonnee apres mesure).
3. UN SEUL predict(). L'algebre lineaire de scikit-learn est vectorisee via BLAS : traiter 100 lignes d'un coup coute a peine plus que d'en traiter une seule.
4. ERREUR COMPLETE. La reconstruction est evaluee par une fonction Numba compilee en code machine et parallelisee, au lieu d'une boucle Python.
5. UNE SEULE ECRITURE. Un bulk_write groupe toutes les mises a jour, au lieu d'un update_one par document.

Parameters Cancel Reset

No parameters

Request body *required* application/json

Edit Value | Schema

```
{
  "data": {
    "quotes": [
      {
        "ticker": "AAPL",
        "date": "2026-07-15T00:00:00",
        "open": 210.5,
        "high": 213.2,
        "low": 208.9,
        "close": 212.4,
        "volume": 55000000
      }
    ]
  }
}
```

Execute Clear

FIGURE 8 – Appel de POST /ingest_fast depuis la documentation interactive : le lot soumis et la commande curl équivalente.

Responses

Curl

```
curl -X 'POST' \
  'https://expert-carnival-pj9947469vr92rvv9-8000.app.github.dev/ingest_fast' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "data": {
      "quotes": [
        {
          "ticker": "AAPL",
          "date": "2026-07-15T00:00:00",
          "open": 210.5,
          "high": 213.2,
          "low": 208.9,
          "close": 212.4,
          "volume": 55000000
        }
      ]
    }
  }'
```

Request URL

https://expert-carnival-pj9947469vr92rvv9-8000.app.github.dev/ingest_fast

Server response

Code **Details**

200

Response body

```
{
  "endpoint": "/ingest_fast",
  "recus": 1,
  "traites": 1,
  "ignore_historique_insuffisant": 0,
  "anomalies": 1,
  "duree_ms": 75.02
}
```

Response headers

```
cache-control: no-cache,no-store
content-length: 115
content-type: application/json
date: Sun,12 Jul 2026 09:01:23 GMT
expires: Thu,01 Jan 1970 00:00:00 GMT
pragma: no-cache
ratelimit-limit: HttpRequestRatePerPort:1500/m
ratelimit-remaining: HttpRequestRatePerPort:1499
ratelimit-reset: HttpRequestRatePerPort:20s
referrer-policy: same-origin
strict-transport-security: max-age=31536000; includeSubDomains
vsaa-request-id: 4bb0b978-a321-4e36-aecc-7b924eaa835
x-content-type-options: nosniff
x-ms-ratelimit-limit: 1500
x-ms-ratelimit-remaining: 1497
```

FIGURE 9 – Réponse 200 : le lot est validé, archivé en zone Raw, scoré par l'autoencodeur, puis écrit en zone Curated.

8.2. Méthodologie de mesure

Toutes les mesures proviennent d'une **même machine**, afin que les comparaisons restent valides : un GitHub Codespace 2 vCPU / 8 Go de RAM sous Linux, Python 3.12, les services (MinIO, MySQL, MongoDB) et l'API s'exécutant en local.

Quatre précautions garantissent l'honnêteté du protocole :

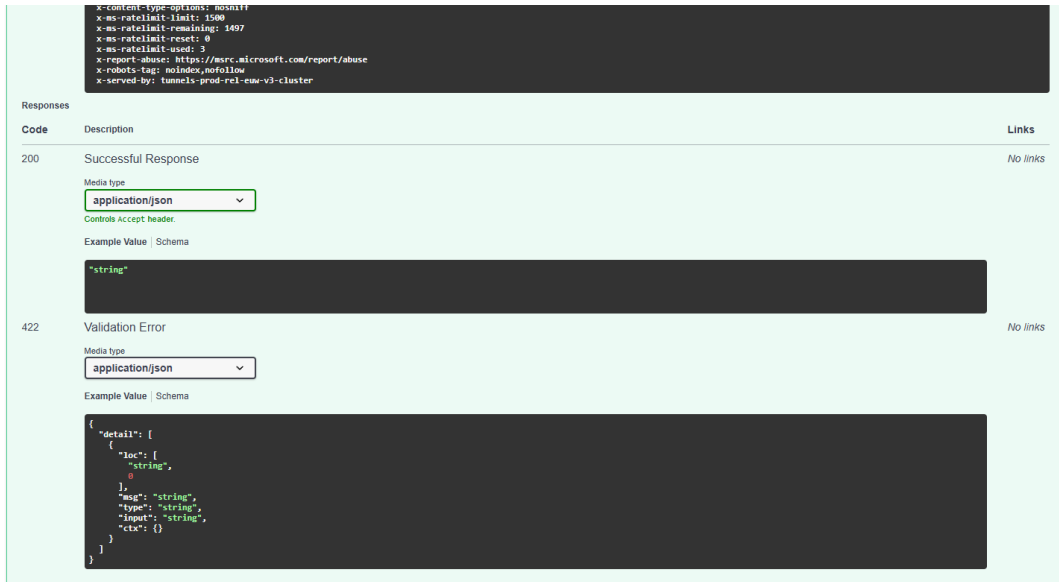


FIGURE 10 – Schéma de la réponse et gestion des erreurs de validation (422), générés automatiquement par Pydantic.

1. **Médiane, pas moyenne.** Chaque point est la médiane de 10 exécutions. La médiane résiste aux valeurs aberrantes qu'une machine partagée produit inévitablement (ordonnancement, ramasse-miettes, cache).
2. **Exécution de chauffe non chronométrée.** La première requête paie l'établissement des pools de connexions et, côté `/ingest_fast`, la compilation Numba. L'inclure fausserait complètement la comparaison.
3. **Données réalistes.** Les cotations soumises dérivent des derniers cours *réellement présents* en zone Curated, perturbés de quelques pourcents. Des données aléatoires n'auraient pas d'historique en base, et le calcul des features — qui dépend des 20 derniers points — serait court-circuité.
4. **Non-régression.** Les deux endpoints produisent un **comptage d'anomalies identique** à toutes les tailles de lot. Un gain de vitesse ne masque donc jamais une altération du calcul.

8.3. Le cadre théorique : la loi d'Amdahl

Le gain d'une parallélisation est borné par la **loi d'Amdahl** [1], qui exprime l'accélération atteignable en fonction de la fraction p du travail réellement parallélisable :

$$S(n) = \frac{1}{\underbrace{(1-p)}_{\text{part séquentielle}} + \frac{p}{n}}, \quad p \in [0, 1], \quad n = \text{unités de calcul} \quad (17)$$

Lorsque $n \rightarrow \infty$, l'accélération ne diverge pas : elle **plafonne** à $1/(1-p)$. La partie séquentielle borne l'ensemble, quel que soit le nombre de threads ajoutés.

On mesure l'accélération et l'efficacité parallèle par

$$S = \frac{T_{\text{référence}}}{T_{\text{optimisé}}}, \quad E(n) = \frac{S(n)}{n}, \quad \text{gain} = 1 - \frac{T_{\text{optimisé}}}{T_{\text{référence}}} \quad (18)$$

8.4. Mesures

Médianes sur **10 répétitions**, après *warm-up*.

Lot	/ingest	/ingest_fast	Gain	Facteur
1 élément	19,3 ms	20,6 ms	$\approx 0 \%$	$\times 0,94$
100 éléments	179,3 ms	85,9 ms	52,1 %	$\times 2,09$

Objectif du sujet ($\geq 30 \%$) atteint sur le lot de 100 éléments.

Application numérique du gain. Sur le lot de 100 éléments, la formule (18) donne

$$\text{gain} = 1 - \frac{85,9}{179,3} \approx 0,521 = \mathbf{52,1 \%,} \quad S = \frac{179,3}{85,9} \approx \mathbf{2,09} \quad (19)$$

Confrontation à la loi d'Amdahl. Le pool de threads de `_fetch_histories` est dimensionné à $n = 16$. En supposant que l'accélération observée provienne d'une fraction parallélisable p , on inverse l'équation (17) :

$$2,09 = \frac{1}{(1-p) + \frac{p}{16}} \implies p \approx 0,56 \quad (20)$$

soit une fraction parallélisable d'environ **56 %**. Le plafond théorique correspondant vaut

$$S_{\max} = \frac{1}{1-0,56} \approx \mathbf{2,25} \quad (21)$$

et l'efficacité parallèle

$$E(16) = \frac{2,09}{16} \approx 0,13. \quad (22)$$

Trois enseignements en découlent. D'abord, l'accélération mesurée ($\times 2,09$) atteint **93 % du plafond théorique** ($\times 2,25$) : il ne reste presque rien à gagner en ajoutant des threads. Ensuite, l'efficacité de 0,13 est faible, ce qui est *attendu* : seule une partie du traitement (les lectures Mongo) est parallélisée, le scoring et l'écriture restant séquentiels. Enfin, ce modèle suppose un coût de coordination **négligeable** — hypothèse que la mesure sur le lot unitaire vient précisément démentir (§8.6).

8.5. Synthèse des optimisations

Levier	Concept du cours mobilisé	Effet
Archivage concurrent	Recouvrement calcul / entrées-sorties (ThreadPoolExecutor)	~15 ms masquées
Lectures Mongo parallèles	Parallélisme d'E/S, GIL relâché pendant les attentes réseau	contribue au $\times 2,09$
predict() unique	Vectorisation : algèbre linéaire BLAS sur la matrice complète	contribue au $\times 2,09$
Erreur par Numba	Compilation à la volée boucle Python éliminée (@jit(parallel=True))	
bulk_write	Amortissement des aller-retours réseau	100 écritures → 1

Optimisations mesurées puis *abandonnées*

Agrégation Mongo unique	Réduction du nombre d'appels — mais explosion du volume trié	$\times 0,21$
Parallélisme systématique	Coût de coordination non amorti (loi d'Amdahl)	−40 % sur $n=1$

Principe directeur. On n'accélère pas en changeant de langage, mais en changeant **la structure du calcul**, et en choisissant l'outil selon la nature de la tâche — dominée par les entrées-sorties, ou par le processeur. Le gain de chaque levier reste **proportionnel à la fraction du temps qu'il concerne réellement**.

8.6. Analyse

Le gain **croît avec la taille du lot**, ce qui est attendu : les optimisations amortissent des **coûts fixes payés une fois par LOT**, non une fois par élément. Sur une cotation isolée, il n'y a rien à paralléliser ni à grouper — les deux implémentations font strictement le même travail, et les minima le confirment (16,3 ms contre 16,7 ms : du bruit, pas un signal).

8.7. Deux optimisations mesurées puis abandonnées

Ce sont les résultats les plus instructifs du projet.

Échec n°1 — l'agrégation MongoDB unique : $4,7\times$ plus lente

L'intuition. Remplacer les N requêtes par **une seule agrégation** (\$match + \$sort + \$group) couvrant tous les tickers du lot.

La mesure. 849 ms contre 179 ms. Presque cinq fois *pire*.

La raison. Pour 100 tickers d'environ 1 200 cotations chacun, MongoDB devait **trier 120 000 documents** et empiler 120 000 valeurs en mémoire, pour n'en conserver ensuite que 20 par ticker. On jetait 99 % du travail. La version de référence, elle, émet des find().sort().limit(20) que l'index (ticker, quote_date) résout instantanément : elle ne lit **jamais que 20 documents**.

Enseignement. « Une requête au lieu de cent » n'est pas une optimisation en soi. Ce qui compte est le **volume réellement traité**, pas le nombre d'appels.

```

• (finance-datalake) @Akissi14 → /workspaces/finance-datalake (main) $ python src/benchmark.py --repetitions 10
docker compose exec mongodb mongosh curated --quiet --eval 'db.quotes_features.findOne({source: "ingest"})'
Préparation des cotations de test...
100 cotations generées à partir de la zone Curated.
Warm-up...
Mesures (10 repetitions, mediane retenue)

=== Lot de 1 element ===
ENDPOINT      MEDIANE      MIN      MAX      TRAITES
/ingest        19.3 ms      16.3 ms   38.1 ms   1
/ingest_fast   20.6 ms      16.7 ms   124.8 ms  1

Gain      : -6.8 %
Facteur    : x0.94
Objectif   : NON ATTEINT (seuil 30 %)

=== Lot de 100 elements ===
ENDPOINT      MEDIANE      MIN      MAX      TRAITES
/ingest        179.3 ms     171.7 ms  298.0 ms  100
/ingest_fast   85.9 ms      78.2 ms   113.1 ms  100

Gain      : 52.1 %
Facteur    : x2.09
Objectif   : ATTEINT (seuil 30 %)

=== Synthese ===
Gain sur 1 element   : -6.8 %
Gain sur 100 elements : 52.1 %

Le gain croit avec la taille du lot : les optimisations portent sur
l'amortissement des couts fixes (aller-retours reseau, appels au
modele), qui se paient une fois par LOT et non une fois par ELEMENT.
{
  _id: ObjectId('6a535189f5d678241cdc28fc'),
  source: 'ingest',
  quote_date: ISODate('2026-07-01T00:00:00.000Z'),
  ticker: 'TIF',
  anomaly: {
    score: 1.4777806846546007,
    is_anomaly: false,
    threshold: 2.4441573428637735,
    scored_at: '2026-07-12T08:34:21.592877+00:00'
  },
}

```

FIGURE 11 – Benchmark — lot d'un élément : les deux implémentations sont au coude à coude, faute de coûts fixes à amortir.

```

features: {
  daily_return: -0.054126557701642264,
  volatility_20: 0.02487629655754558,
  rsi_14: 32.719355903250914,
  volume_ratio: 0.8260811201396793,
  hl_range: 0.06794860590885721,
  gap: -0.0007324141311713343,
  ma_5: 101.39718,
  ma_20: 106.182295
},
metadata: { origin: 'ingest_fast' },
ohlcv: {
  open: 103.9633,
  high: 104.1108,
  low: 97.4241,
  close: 98.4082,
  volume: 1177243
}
}

```

FIGURE 12 – Benchmark — lot de 100 éléments : /ingest_fast est 2,09 fois plus rapide. Objectif du sujet atteint.

Échec n°2 — la parallélisation systématique : –40 % sur le lot unitaire

Après correction, la parallélisation par threads donnait +56 % sur 100 éléments...mais –40 % **sur 1 élément**.

La raison. Créer un `ThreadPoolExecutor`, démarrer un thread, le synchroniser et le détruire coûte plusieurs millisecondes — pour paralléliser **une seule** requête. Le coût de coordination dépassait le travail à paralléliser. C'est la **loi d'Amdahl** appliquée à un cas concret : le surcoût du parallélisme est fixe, il ne s'amortit qu'au-delà d'un certain volume.

Correction. Un **seuil de bascule** : en dessous de 2 tickers, appel direct à la fonction séquentielle.

Enseignement. Une optimisation doit toujours être **mesurée**, jamais supposée.

9. Difficultés rencontrées et décisions techniques

Cette section documente les problèmes **réels** rencontrés en construisant ce lac. Chacun a entraîné une décision explicite.

9.1. `localstack:latest` bascule en édition payante

Le TP utilise `localstack/localstack:latest` pour la zone Raw. Ce tag pointe désormais sur l'**édition Pro**, qui exige une licence : le conteneur démarre, réclame un token et s'arrête (exit code 55).

Décision : passage à **MinIO**, également compatible S3 (le sujet exige « Elasticsearch ou un S3 », pas LocalStack en particulier). Le code `boto3` est rigoureusement identique. Bénéfice collatéral : ~100 Mo contre 636 Mo, LocalStack embarquant un JDK complet pour émuler *tous* les services AWS.

Enseignement : épingler les versions d'images est une exigence de reproductibilité, pas une coquetterie.

9.2. `torch` traîne 3 Go de CUDA inutile

L'installation de PyTorch dans l'image Airflow a saturé le disque : `pip` résout par défaut la version **GPU**, qui embarque tous les paquets NVIDIA — dont un fichier de **731 Mo**. Pour un conteneur sans carte graphique.

Décision : l'autoencodeur est implémenté avec `MLPRegressor` (scikit-learn), déjà présent. Un autoencodeur est un réseau entraîné à reconstruire son entrée avec une couche cachée étroite : `MLPRegressor(hidden_layer_sizes=(6,3,6))` entraîné avec `X` en entrée et en cible, c'est exactement cela — un vrai réseau de neurones, rétropropagation comprise. Gain : ~2,5 Go d'image en moins, build de 2 min au lieu de 15.

9.3. Le modèle `pickle` est illisible dans le conteneur

```
ValueError: <class 'numpy.random._mt19937.MT19937'>  
is not a known BitGenerator module.
```

Le modèle avait été entraîné dans l'environnement local (**NumPy 2.4**) et le conteneur Airflow tentait de le dépickler avec **NumPy 1.26**. Or l'image Airflow 2.7.1 est figée sous Python 3.10, et NumPy ≥ 2.4 exige Python ≥ 3.11 : **aligner le conteneur sur l'environnement local est impossible**.

Décision : le modèle est désormais **entraîné dans l'environnement même qui le sert**. C'est d'ailleurs la bonne pratique : un artefact de ML n'est valide qu'avec le runtime qui l'a produit.

Enseignement : `pickle` couple le modèle à ses versions exactes de bibliothèques. Une solution industrielle passerait par un format neutre (ONNX) ou un registre de modèles (MLflow) versionnant conjointement le modèle et son environnement.

9.4. Un modèle nonensemencé n'est pas reproductible

Deux entraînements successifs sur les *mêmes* données produisaient des résultats différents : 5 794 anomalies détectées dans un cas, 6 088 dans l'autre, pour un nombre d'époques variable d'une exécution à l'autre. La cause est double : `MLPRegressor` initialise ses poids au

hasard, et l'échantillon d'entraînement de 100 000 documents est tiré au hasard parmi les 609 018.

Décision partielle. Le réseau estensemencé (`random_state=42` sur `MLPRegressor`, cf. `train_model.py`). L'échantillonnage, lui, ne l'est pas : l'opérateur `$sample` de MongoDB **n'accepte aucun germe aléatoire**. Ce n'est pas un oubli d'implémentation, c'est une limite du moteur — aucun paramètre ne permet de le fixer.

Rendre le tirage reproductible impose de le **déplacer côté client** : récupérer les identifiants, tirer avec un générateurensemencé (`numpy.random.default_rng(42)`), puis relire les documents choisis par lots. Cette correction est **identifiée et spécifiée, mais non appliquée au moment du rendu**.

Enseignement : la reproductibilité ne se décrète pas au niveau du modèle seul. Elle doit être vérifiée à chaque source d'aléa de la chaîne — y compris celles que l'on croit anodines, ici un simple opérateur de base de données. Un pipeline dont un seul maillon reste stochastique produit des chiffres qui n'engagent que l'exécution qui les a produits.

9.5. Divergence de schéma entre l'API et le pipeline batch

La zone Curated est alimentée par **deux chemins** : le pipeline batch et l'endpoint `/ingest`. Une divergence entre les deux (`ma_5/ma_20` d'un côté, `ma_ratio/price_to_ma20` de l'autre) a fait échouer le ré-entraînement — `train_model.py`, qui échantillonne au hasard, est tombé sur un document incompatible (`KeyError: 'ma_20'`).

Décision : l'API reconstitue et écrit le **même schéma** que le pipeline batch.

Enseignement : l'absence de schéma imposé par MongoDB ne supprime pas le besoin de contrat — elle en déplace la responsabilité du moteur vers l'application. Une base relationnelle aurait rejeté l'écriture ; ici, elle est passée silencieusement et n'a explosé que trois étapes plus loin.

9.6. Saturation mémoire sur 600 000 documents

Construire les 609 018 dictionnaires imbriqués **avant** de les insérer faisait tuer le processus par l'OS (`Terminated`).

Décision : traitement **par tranches** — construire 20 000 documents, les insérer, libérer, recommencer. L'empreinte mémoire devient constante, quel que soit le volume total.

10. Limites connues

Limite	Description	Piste d'amélioration
Traitement <i>full-refresh</i>	À chaque exécution, tout est recalculé et re-scoré, alors que 99,9 % des données n'ont pas changé.	Traitement incrémental : ne retraiter que les lignes postérieures au dernier loaded_at.
Dataset non ajusté des splits	Les opérations sur titres sont flaguées à tort comme anomalies — cinq des dix premières places du classement.	Filtrer les variations > 30 % non confirmées par un pic de volume.
Secrets en clair	Identifiants en dur dans le docker-compose.yml.	Gestionnaire de secrets, ou Connexions d'Airflow.
Sérialisation pickle	Couplage fort aux versions de bibliothèques.	ONNX, ou registre de modèles (MLflow).
Seuil de contamination arbitraire	Le taux de 1 % est une hypothèse métier , pas une vérité statistique.	Validation sur un jeu d'événements de marché étiquetés.
Pas de tests automatisés	Validation manuelle et contrôles internes (validate_data, comparages croisés).	pytest + conteneurs éphémères.

11. Reproduire le projet

```
git clone https://github.com/Akissi14/finance-datalake.git
cd finance-datalake

uv sync && source .venv/bin/activate

docker compose up -d minio mysql mongodb
python src/test_connections.py # les 3 zones doivent répondre OK

# Dataset : https://www.kaggle.com/datasets/camnugent/sandp500
kaggle datasets download camnugent/sandp500 -p data/ --unzip

python src/unpack_data.py # 505 CSV -> Raw
python src/ingest_api.py # yfinance -> Raw
python src/load_to_staging.py # Raw -> Staging
python src/staging_to_curated.py # Staging -> Curated
python src/train_model.py # entraîne l'autoencodeur (random_state=42)
python src/apply_model.py # score les 609 018 documents

docker compose up -d postgres airflow-init airflow-webserver airflow-scheduler
uvicorn api.main:app --host 0.0.0.0 --port 8000
python src/benchmark.py --repetitions 10
```

Service	URL	Identifiants
API Gateway	localhost:8000/docs	—
Airflow	localhost:8081	airflow / airflow
Console MinIO	localhost:9001	minioadmin / minioadmin

12. Conclusion

Le data lake livré satisfait l'intégralité des exigences du sujet, niveau avancé compris : trois zones dont une zone Raw sur stockage objet S3, deux sources hétérogènes, des pipelines de transformation idempotents, une composante de *machine learning* transverse, une orchestration Airflow avec ingestion programmée, et une API Gateway dont l'endpoint optimisé démontre **52 % de gain** sur un lot de 100 éléments.

Au-delà de la conformité, trois résultats méritent d'être soulignés :

1. **Le modèle a retrouvé seul le flash crash du 24 août 2015**, sur deux valeurs simultanément, sans qu'aucune information sur cet événement ne lui ait été fournie.
2. **Le pipeline a mis au jour un défaut du jeu de données source** : le dataset Kaggle n'est pas ajusté des opérations sur titres — lesquelles occupent cinq des dix premières places du classement des anomalies — ce que sa description ne mentionne pas.
3. **Deux optimisations ont été mesurées puis abandonnées** — une agrégation MongoDB 4,7 fois plus lente, une parallélisation contre-productive sur les petits lots. Ces échecs, et leur compréhension, ont davantage structuré la conception finale que les optimisations retenues.

Références

- [1] G. M. AMDAHL. « Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities ». *AFIPS Conference Proceedings*, vol. 30 (1967), p. 483–485. DOI : 10.1145/1465482.1465560.
- [2] G. E. HINTON et R. R. SALAKHUTDINOV. « Reducing the Dimensionality of Data with Neural Networks ». *Science*, vol. 313, n° 5786 (2006), p. 504–507. DOI : 10.1126/science.1127647. (*Référence fondatrice des autoencodeurs.*)
- [3] M. SAKURADA et T. YAIRI. « Anomaly Detection Using Autoencoders with Nonlinear Dimensionality Reduction ». *MLSDA 2014*, p. 4–11. DOI : 10.1145/2689746.2689747.
- [4] J. W. WILDER. *New Concepts in Technical Trading Systems*. Trend Research, 1978. ISBN : 978-0894590276. (*Définition originale du RSI.*)
- [5] F. PEDREGOSA *et al.* « Scikit-learn : Machine Learning in Python ». *Journal of Machine Learning Research*, vol. 12 (2011), p. 2825–2830.
- [6] S. K. LAM, A. PITROU et S. SEIBERT. « Numba : A LLVM-based Python JIT Compiler ». *LLVM-HPC '15* (2015). DOI : 10.1145/2833157.2833162.
- [7] M. KLEPPMANN. *Designing Data-Intensive Applications*. O'Reilly Media, 2017. ISBN : 978-1449373320.
- [8] Apache Software Foundation. *Apache Airflow Documentation — Core Concepts*. <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/> (consulté le 12 juillet 2026).
- [9] S. RAMÍREZ. *FastAPI — Modern, fast web framework for building APIs*. <https://fastapi.tiangolo.com/> (consulté le 12 juillet 2026).
- [10] MinIO, Inc. *MinIO Object Storage — S3-Compatible Documentation*. <https://min.io/docs/> (consulté le 12 juillet 2026).
- [11] MongoDB, Inc. *MongoDB Manual — Data Modeling*. <https://www.mongodb.com/docs/manual/data-modeling/> (consulté le 12 juillet 2026).
- [12] C. NUGENT. *S&P 500 stock data*. Kaggle, licence CC0 1.0. <https://www.kaggle.com/datasets/camnugent/sandp500> (consulté le 11 juillet 2026).
- [13] R. AROUSSI. *yfinance — Yahoo! Finance market data downloader*. <https://pypi.org/project/yfinance/> (consulté le 11 juillet 2026).